Lazy & IO

Lazy/IO · Defer expensive report until edge · Build with Waqar

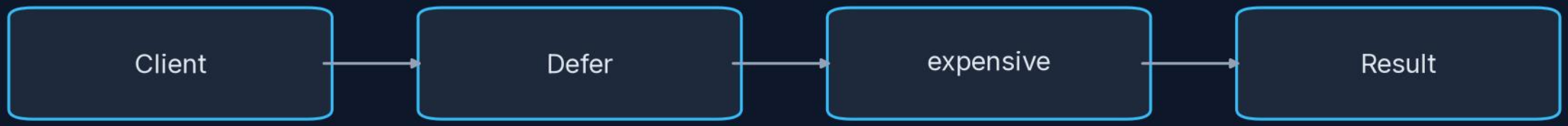
Today's Hook

- Today's force: Defer expensive report until edge
- We earn Lazy/IO by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: Lazy, IO

The Problem

- Scenario: Defer expensive report until edge
- Constraints: solve «Defer expensive report until edge» with Lazy/IO — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Defer expensive report until edge
- Success looks like: BUILD

Structure



Lazy/IO: Defer expensive report until edge

Real-World Scenario

- Running example: Defer expensive report until edge
- Key types: Lazy, IO
- Happy-path stdout cue: BUILD
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```
package com.buildwithwaqar.patterns.functional.lazy.problem;

import java.util.*;

public final class NaiveReport {
  public static String build(){ System.out.println("BUILD"); return "report"; }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Defer expensive report until edge
- Without Lazy/IO, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```
package com.buildwithwaqar.patterns.functional.lazy.problem;

import java.util.*;
import java.util.function.*;

final class Lazy<A> {
  private final Supplier<A> thunk; private A value; private boolean ready;
  Lazy(Supplier<A> thunk){ this.thunk=thunk; }
  static <A> Lazy<A> of(Supplier<A> s){ return new Lazy<>(s); }
  synchronized A get(){ if(!ready){ value=thunk.get(); ready=true; } return value; }
}
final class IO<A> {
  private final Supplier<A> effect; IO(Supplier<A> effect){ this.effect=effect; }
  static <A> IO<A> delay(Supplier<A> s){ return new IO<>(s); }
  <B> IO<B> map(Function<A,B> f){ return new IO<>(() -> f.apply(effect.get())); }
  A unsafeRun(){ return effect.get(); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.lazy.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Lazy/IO: defer expensive report ===");
    System.out.println("--- naive ---"); System.out.println(NaiveReport.build());
System.out.println("--- lazy ---");
var lazy=Lazy.of(() -> { System.out.println("BUILD"); return "report"; });
System.out.println("created"); System.out.println(lazy.get()); System.out.println(lazy.get());
System.out.println("--- io ---");
System.out.println(IO.delay(() -> "edge").map(s -> s+"!").unsafeRun());
  }
}
```

Walkthrough Output

```
=== Lazy/IO: defer expensive report ===
--- naive ---
BUILD
report
--- lazy ---
created
BUILD
report
report
--- io ---
edge!
```

Step-by-Step Trace

- Match stdout to stages — look for: BUILD
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp … 46-lazy-io ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Lazy/IO
// Prefer records + sealed where they shrink the demo.
// Problem under test: Defer expensive report until edge
```

Trade-offs

- Use Lazy/IO when «Defer expensive report until edge» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Lazy/IO in product language.

Related Patterns

- Name the pattern usually confused with Lazy/IO and separate the forces.
- Sketch the roles for «Defer expensive report until edge» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Defer expensive report until edge
- Naive pain → Lazy/IO boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.